Análisis asintótico, Dividir para Conquistar *y un k-d tree*

Paula Vásquez-Henríquez · Guillermo Zieballe Profesora Cecilia Hernández · 13 de octubre de 2026

Doctorado en Inteligencia Artificial · CRUCH
Biobío

Contenido

1 **Contar operaciones en ciclos**
Algoritmos A, B y C

2 **Demostrar cotas por definición**
Constantes c_1, c_2 y n_0 explícitas

3 **Resolver recurrencias**
Árbol de recursión y sustitución

4 **k-d tree para clasificación 1-NN**
Implementación y experimento con PCA

ACTIVIDAD

1 Contar operaciones en ciclos

```
Procedure A(n)
  for i = 1 to n
    j = i
    while j <= n
      Operation()
      j = 2*j
```

ACTIVIDAD 1 · ALGORITMO A

Un for y un while que duplica. ¿Es $\Theta(n \log n)$?

Para un i fijo, el while da $\lfloor \log_2(n/i) \rfloor + 1$ vueltas

después de m vueltas

$$j = i \cdot 2^m$$

se da otra vuelta si

$$i \cdot 2^m \le n \iff m \le \log_2(n/i)$$

como m es entero, toma los valores

$$m = 0, 1, \ldots, \lfloor \log_2(n/i) \rfloor$$

c(i) = cuántos valores toma m

$$c(i) = \lfloor \log_2(n/i) \rfloor + 1$$

(b) $T(n) = c(1) + c(2) + \cdots + c(n)$

i	valores de j (n = 8)	vueltas
1	1, 2, 4, 8	4
2	2, 4, 8	3
3	3, 6	2
4	4, 8	2
5 a 8	solo i	1 cada uno
Total		15

ACTIVIDAD 1 · ALGORITMO A · (C)

Contamos cuántos *i* llegan a cada iteración del while

iteración k	i que llegan (n = 8)	cantidad
1	1, 2, …, 8	8
2	1, 2, 3, 4	4
3	1, 2	2
4	1	1
Total		15

a la iteración k llegan $\lfloor n / 2^{k-1} \rfloor$ valores de i

$$T(n) = n + \lfloor n/2 \rfloor + \lfloor n/4 \rfloor + \cdots$$

sin pisos y con la suma infinita

$$T(n) \le n\,(1 + \tfrac{1}{2} + \tfrac{1}{4} + \cdots) = 2n$$

cota inferior n y cota superior 2n

$$n \le T(n) \le 2n \Rightarrow T(n) \in \Theta(n)$$

$c_1 = 1$, $c_2 = 2$, $n_0 = 1$

ALGORITMO A · (D)

Multiplicar n por las vueltas del while solo sirve si todos los i dan *las mismas vueltas*.

Con n = 8, i = 1 da 4 vueltas e i = 5, 6, 7, 8 dan una. Además $T(n) \leq 2n$ crece más lento que n log n.

```
Procedure B(n)
  m = n
  while m >= 1
    for i = 1 to m
      j = 1
      while j <= m
        Operation()
        j = 2*j
    m = floor(m/2)
```

ACTIVIDAD 1 · ALGORITMO B

El mismo while, pero *j parte en 1*. ¿Cuánto cambia?

En B el while siempre parte en 1, y ahí sí se multiplica

Algoritmo A: `j = i`

Cada i da distinto número de vueltas. Con n = 8, i = 1 da 4 vueltas e i = 5, 6, 7, 8 dan una.

Hay que sumar vuelta por vuelta.

$$T(n) \in \Theta(n)$$

Algoritmo B: `j = 1`

Para un m fijo, todos los i dan $\lfloor \log_2 m \rfloor + 1$ vueltas.

W(m) cuenta las Operation() de la pasada del while externo con ese m:

$$W(m) = m\,(\lfloor \log_2 m \rfloor + 1)$$

cota inferior $n \log_2 n$ y cota superior $4\,n \log_2 n$

$$n \log_2 n \le T(n) \le 4\,n \log_2 n$$

$c_1 = 1$, $c_2 = 4$, $n_0 = 2 \Rightarrow T(n) \in \Theta(n \log n)$

La suma de A reaparece en la cota superior de B

(b) m toma los valores n, ⌊n/2⌋, ⌊n/4⌋, …

$$T(n) = W(n) + W(\lfloor n/2 \rfloor) + W(\lfloor n/4 \rfloor) + \cdots$$

Con n = 8:

$$32 + 12 + 4 + 1 = 49$$

comprobación de las cotas con n = 8

$$8 \cdot 3 = 24 \le 49 \le 96 = 4 \cdot 8 \cdot 3$$

(c) Cotas

Cota superior

como m ≤ n, cada while interno da a lo más $\log_2 n + 1$ vueltas

$$W(m) \le m\,(\log_2 n + 1)$$

sumamos las pasadas, $\log_2 n + 1$ sale como factor y queda la suma de A

$$T(n) \le (\log_2 n + 1)(n + \lfloor n/2 \rfloor + \cdots) \le 2n\,(\log_2 n + 1)$$

con $n \ge 2$, $1 \le \log_2 n$, así que $\log_2 n + 1 \le 2 \log_2 n$

$$T(n) \le 4\,n \log_2 n$$

Cota inferior

la suma es al menos su primer término, la pasada con m = n

$$T(n) \ge W(n) = n\,(\lfloor \log_2 n \rfloor + 1) \ge n \log_2 n$$

```
Q = X * W_Q
K = X * W_K
V = X * W_V
S = Q * transpose(K)
A = row_softmax(S)
Y = A * V
```

X es de n × d

W_Q, W_K, W_V son de d × d

ACTIVIDAD 1 · ALGORITMO C

Seis líneas con matrices. ¿Cuánto cuestan *en función de n y d*?

ACTIVIDAD 1 · ALGORITMO C

Multiplicar (p × q)(q × r) cuesta Θ(pqr)

$$C[a][b] = P[a][1] \cdot R[1][b] + P[a][2] \cdot R[2][b] + \cdots + P[a][q] \cdot R[q][b]$$

$\Theta(q)$

por posición

q multiplicaciones y q − 1 sumas

$p \cdot r$

posiciones

el resultado es de p × r

$\Theta(pqr)$

en total

dos matrices de n × n: $\Theta(n^3)$

ACTIVIDAD 1 · ALGORITMO C

Costo de cada línea, mirando las dimensiones

línea	dimensiones	costo
$Q, K, V = X \cdot W$	$(n \times d)(d \times d)$, tres veces	$\Theta(nd^2)$
K^T	$(n \times d) \to (d \times n)$	$\Theta(nd)$
$S = Q \cdot K^T$	$(n \times d)(d \times n)$	$\Theta(n^2 d)$
$A = \text{row_softmax}(S)$	n filas de n	$\Theta(n^2)$
$Y = A \cdot V$	$(n \times n)(n \times d)$	$\Theta(n^2 d)$
Total (K^T y softmax quedan absorbidos)		$\Theta(nd^2 + n^2 d)$

Qué término domina y cuánto espacio se usa

Comparamos los dos términos

$$n^2d \,/\, nd^2 = n \,/\, d$$

n ≫ **d**: domina $\Theta(n^2d)$, que viene de S y de Y.

d ≫ **n**: domina $\Theta(nd^2)$, que viene de Q, K y V.

Espacio de las matrices intermedias

$$3nd + nd + 2n^2 \in \Theta(nd + n^2)$$

Q, K, V y K^T ocupan nd cada una. S y A ocupan n^2 cada una.

Escribir las n^2 posiciones de S y A cuesta tiempo, así que el espacio es también un piso para el tiempo.

ACTIVIDAD

2 Demostrar cotas por definición

ACTIVIDAD 2.1

$f(n) = 4n^2 \log_2 n + 3n^2 + 7n + 12 \in \Theta(n^2 \log_2 n)$

$c_1 = 4$

Cota inferior

Quitamos $3n^2$, $7n$ y 12, que son positivos.

$c_2 = 26$

Cota superior

Cada término es a lo más un múltiplo de $n^2 \log_2 n$: $4 + 3 + 7 + 12$.

$n_0 = 2$

Desde dónde vale

La cota superior necesita $\log_2 n \geq 1$.

ACTIVIDAD 2.1

Comprobación: f(n) queda entre las dos cotas

n	$4\,n^2 \log_2 n$	f(n)	$26\,n^2 \log_2 n$
2	16	54	104
4	128	216	832
8	768	1028	4992

ACTIVIDAD 2.2

$n \log_2 n \in O(n^{3/2})$ se reduce a comparar $\log_2 n$ con $\sqrt{n}$

$$n \log_2 n \le c \cdot n\sqrt{n} \iff \log_2 n \le c\sqrt{n}$$

Con c = 1, la desigualdad falla para los enteros de 5 a 15. El n_0 tiene que quedar después del cruce en 16.

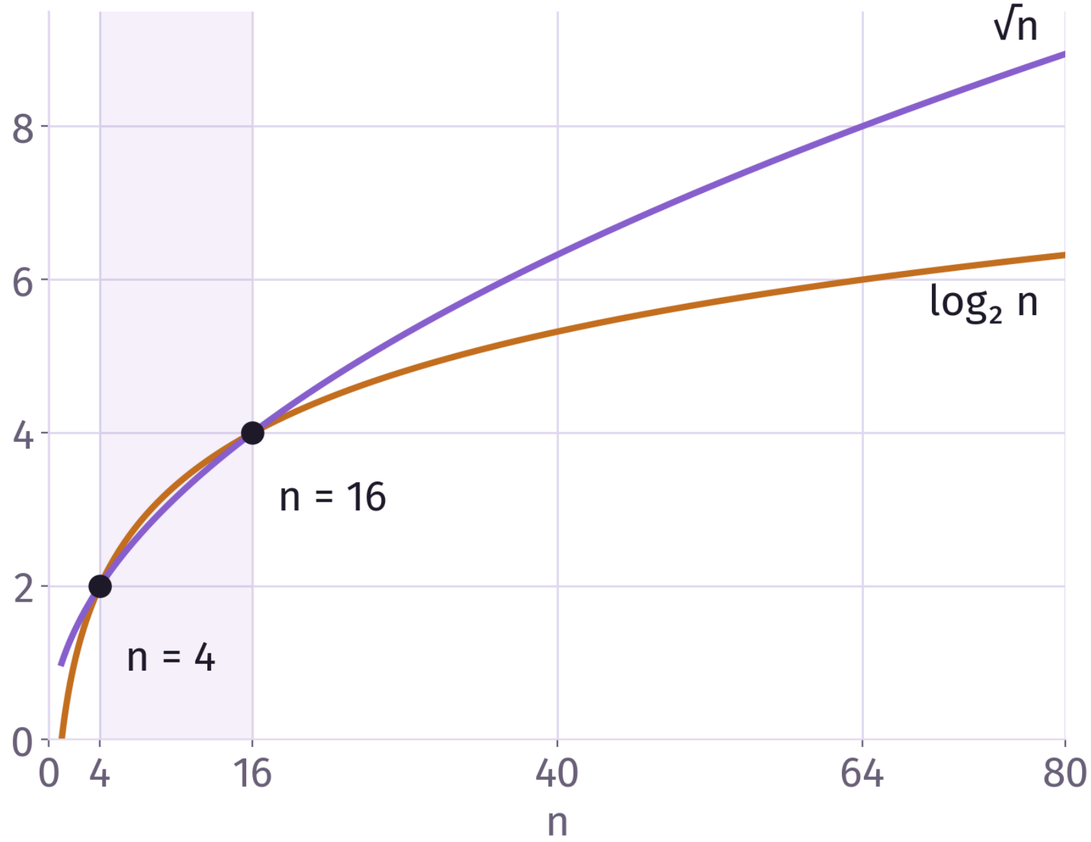


Gráfico generado con presentacion/figuras.py

ACTIVIDAD 2.2

$\log_2 n \leq \sqrt{n}$ desde $n = 64$, por inducción

1

Ubicar n entre potencias de 2

$$2^k \leq n < 2^{k+1}$$

Así $\log_2 n < k + 1$ y $\sqrt{(2^k)} \leq \sqrt{n}$.

Basta probar $(k + 1)^2 \leq 2^k$.

→

2

Inducción desde k = 6

Base: $7^2 = 49 \leq 64 = 2^6$. Con $k = 5$ falla: $36 > 32$.

Paso: si vale para k, vale para $k + 1$.

$$(k+2)^2 = (k+1)^2 + 2k + 3 \leq 2(k+1)^2 \leq 2^{k+1}$$

→

3

Conclusión

Si $n \geq 64$, entonces $k \geq 6$ y $\log_2 n \leq \sqrt{n}$.

$$n \log_2 n \leq 1 \cdot n^{3/2}$$

$c = 1$, $n_0 = 64$

ACTIVIDAD 2.2

Paso inductivo: de $(k+1)^2 \le 2^k$ a $(k+2)^2 \le 2^{k+1}$

HI	$(k+1)^2 \le 2^k$ para algún $k \ge 6$
1	$(k+2)^2 = ((k+1) + 1)^2 = (k+1)^2 + 2k + 3$
2	$(k+1)^2 - (2k+3) = k^2 - 2 > 0$, así que $2k + 3 < (k+1)^2$
3	$(k+2)^2 \le 2(k+1)^2 \le 2 \cdot 2^k = 2^{k+1}$
$\therefore$	Vale para todo $k \ge 6$. El caso $k = 5$ no sirve: $36 > 32$.

ACTIVIDAD

3 Resolver recurrencias

ACTIVIDAD 3.1 · (A)

De RecursiveFilter a su recurrencia

```
Procedure RecursiveFilter(A, n)
  if n <= 1
    return
  for i = 1 to n
    Process(A[i])
  RecursiveFilter(A[1..⌊n/2⌋], ⌊n/2⌋)
  RecursiveFilter(A[1..⌊n/4⌋], ⌊n/4⌋)
```

$$T(n) = c, \text{ si } n \leq 1$$

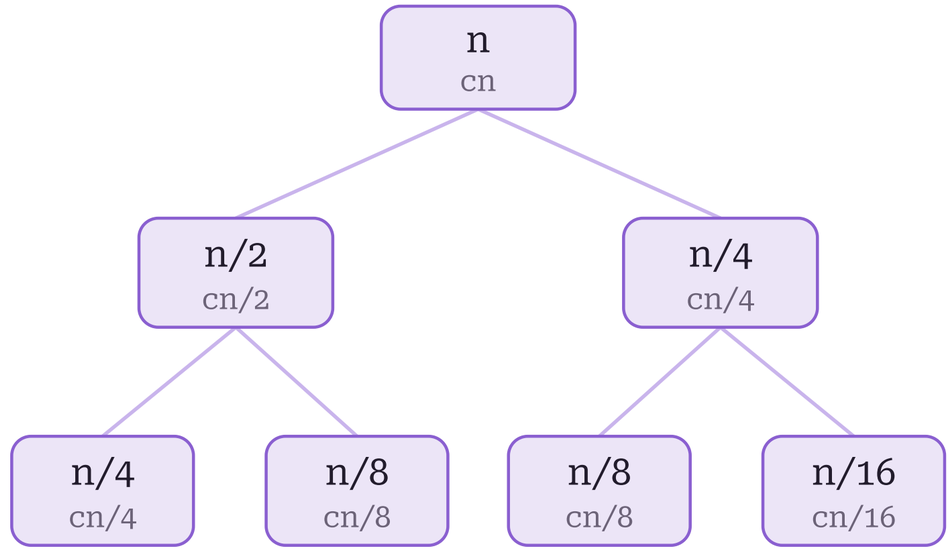
$$T(n) = T(\lfloor n/2 \rfloor) + T(\lfloor n/4 \rfloor) + cn, \text{ si } n > 1$$

El for cuesta $\Theta(n)$. Pasar A[1..k] no copia los datos, cuesta $\Theta(1)$.

Omitimos los pisos. Solo achican los tamaños, así que la cota superior sigue valiendo, y la inferior usa solo la raíz.

ACTIVIDAD 3.1 · (B), (C) Y (D)

T(n) = T(n/2) + T(n/4) + cn: cada nivel hace ¾ del anterior



nivel 0	cn
nivel 1	$(\frac{1}{2} + \frac{1}{4})\, cn = \frac{3}{4}\, cn$
nivel 2	$(\frac{3}{4})^2\, cn$

$$cn\,(1 + \tfrac{3}{4} + (\tfrac{3}{4})^2 + \cdots) = 4cn$$

más las hojas, que cuestan c cada una y suman a lo más 4cn

$$cn \le T(n) \le 8cn \Rightarrow T(n) \in \Theta(n)$$

El trabajo decrece geométricamente y domina la raíz.

ACTIVIDAD 3.1 · (D)

Las hojas se suman aparte porque cuestan c aunque su tamaño sea 0

1

Nodos internos

Tamaño ≥ 2, así que cada uno trabaja al menos 2c. Entre todos suman ≤ 4cn.

a lo más 2n

→

2

Hojas

Cada nodo interno tiene dos hijos.

a lo más 4n

→

3

Costo total

Cada hoja cuesta c.

$T(n) \leq 4cn + 4cn = 8cn$

ACTIVIDAD 3.1 · (E)

Por qué no se aplica el Teorema Maestro

Forma estándar del teorema

$$T(n) = a\,T(n/b) + f(n)$$

Los a subproblemas tienen el mismo tamaño n/b, y se compara f(n) con $n^{\log_b a}$.

Nuestra recurrencia

$$T(n) = T(n/2) + T(n/4) + cn$$

Dos tamaños distintos, así que no hay un único b. Se resuelve con el árbol, como T(n/3) + T(2n/3) + n en clases.

ACTIVIDAD 3.2 · (A)

De BuildKD a su recurrencia

```
BuildKD(P, depth)
  if |P| <= 1
    return leaf(P)
  axis = depth mod d
  sort P according to axis
  m = ⌊|P|/2⌋
  node.point = P[m]
  node.left  = BuildKD(P[0..m-1], depth+1)
  node.right = BuildKD(P[m+1..], depth+1)
  return node
```

$$T(n) = c, \text{ si } n \leq 1$$

$$T(n) = 2T(n/2) + cn \log_2 n, \text{ si } n \geq 2$$

El ordenamiento cuesta $\Theta(n \log n)$ y absorbe las operaciones constantes. Las dos llamadas reciben a lo más $n/2$ puntos.

Suponemos que n es potencia de 2.

ACTIVIDAD 3.2 · (B)

$T(n) = 2T(n/2) + cn \log_2 n$: el trabajo baja de a cn por nivel

nivel	nodos	trabajo del nivel
0	1	$cn \log_2 n$
1	2	$cn (\log_2 n - 1)$
i	2^i	$cn (\log_2 n - i)$
$\log_2 n$	n hojas	cn

sumamos los niveles, con $L = \log_2 n$

$$cn\,L + cn\,(L-1) + \cdots + cn = cn\,(1 + 2 + \cdots + L)$$

suma de Gauss: $1 + 2 + \cdots + L = L(L+1)/2$

$$cn \cdot L(L+1)/2 \approx (c/2)\, n \log_2^2 n$$

con las hojas, el término que manda es el primero

$$(c/2)\, n \log_2^2 n + (c/2)\, n \log_2 n + cn$$

Conjetura

$$T(n) \in \Theta(n \log^2 n)$$

Suponemos la cota para n/2 y comprobamos que vale para n

Cota superior

$$T(n) \leq 2c\, n \log_2^2 n$$

suponemos la cota para n/2 y la reemplazamos

$$T(n) \leq 2 \cdot 2c\,(n/2) \log_2^2(n/2) + cn \log_2 n$$

$\log_2(n/2) = \log_2 n - 1$, desarrollamos el cuadrado

$$= 2c\, n \log_2^2 n - 3cn \log_2 n + 2cn$$

lo que sobra es negativo: con $n \geq 2$, $3cn \log n \geq 3cn > 2cn$

$$-3cn \log_2 n + 2cn \leq 0 \Rightarrow T(n) \leq 2c\, n \log_2^2 n$$

Base: $n = 2$ (próxima lámina).

Cota inferior

$$T(n) \geq (c/2)\, n \log_2^2 n$$

suponemos la cota para n/2 y la reemplazamos

$$T(n) \geq 2 \cdot (c/2)(n/2) \log_2^2(n/2) + cn \log_2 n$$

$\log_2(n/2) = \log_2 n - 1$, desarrollamos el cuadrado

$$= (c/2)\, n \log_2^2 n + (c/2)\, n$$

lo que sobra es positivo

$$(c/2)\, n \geq 0 \Rightarrow T(n) \geq (c/2)\, n \log_2^2 n$$

Base: $T(1) = c \geq 0 = (c/2) \cdot 1 \cdot \log_2^2 1$.

ACTIVIDAD 3.2 · (C)

El caso base en n = 2 fija la constante 2c

n = 1 no sirve

$$2c \cdot 1 \cdot (\log_2 1)^2 = 0$$

Pero T(1) = c > 0. La cota es falsa en n = 1.

n = 2 fija la constante

$$T(2) = 4c \leq 2d$$

Hace falta d ≥ 2c. Tomamos d = 2c.

Alcanza para todo n

$$2 \to 4 \to 8 \to 16 \to \cdots$$

El paso lleva la cota de n/2 a n, y parte en n = 4 para usar la hipótesis.

ACTIVIDAD 3.2 · (D)

Con la mediana en tiempo lineal se ahorra un factor log n

Ordenando en cada llamada

$$T(n) = 2T(n/2) + cn \log n$$

$$\Theta(n \log^2 n)$$

Mediana y partición en Θ(n)

$$T(n) = 2T(n/2) + cn$$

$$\Theta(n \log n)$$

Es la recurrencia de Merge Sort. La mediana lineal se obtiene con mediana de medianas: $T(n/5) + T(7n/10) + n \in \Theta(n)$.

ACTIVIDAD

4 k-d tree para clasificación 1-NN

ACTIVIDAD 4 · PROTOCOLO

Las dos búsquedas pagan lo mismo por cada distancia

1

Datos

digits: 1797 imágenes de 8 × 8. Entrenamiento 1437, prueba 360. Estandarizar y PCA con d = 2, 4, 8, 16, 32.

→

2

Dos búsquedas

k-d tree (mediana por eje, 11 niveles) y búsqueda exhaustiva. Python puro y la misma `squared_distance`.

→

3

Medición

360 consultas por d. Mediana de 7 repeticiones. Nodos visitados = distancias evaluadas.

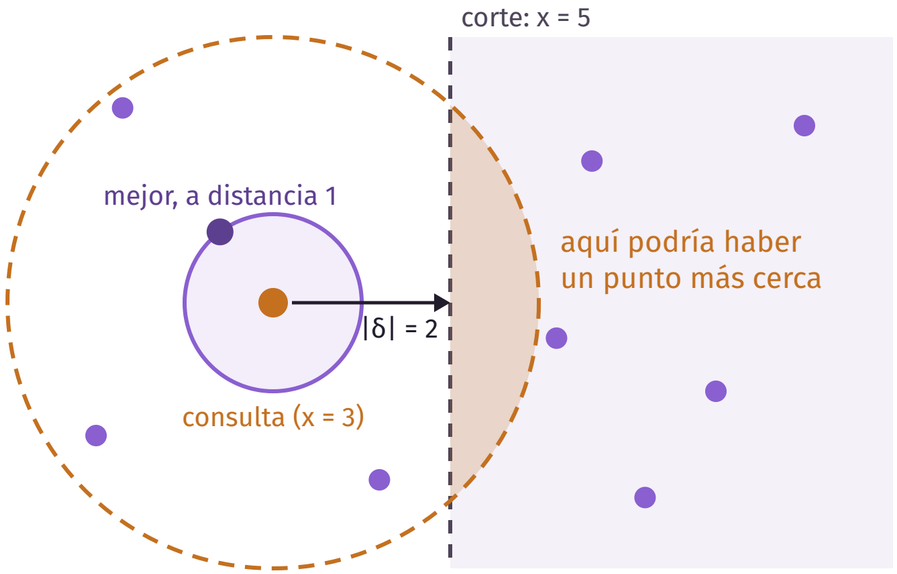
ACTIVIDAD 4 · IMPLEMENTACIÓN

La construcción es el BuildKD de la Actividad 3.2

```
def squared_distance(a, b):
    suma = 0.0
    for j in range(len(a)):
        diff = a[j] - b[j]
        suma = suma + diff * diff
    return suma
```

- Cada nodo es un diccionario: punto, índice, eje y dos hijos.
- Eje = depth mod d. Se ordena por ese eje y la mediana $\lfloor n/2 \rfloor$ queda en el nodo.
- Construcción en $\Theta(n \log^2 n)$. Con 1437 puntos, $\lfloor \log_2 1437 \rfloor + 1 = 11$ niveles.

El otro lado se revisa solo si puede tener un punto más cerca




```
eje = nodo["eje"]
delta = q[eje] - nodo["punto"][eje]
if delta < 0:
    cerca = nodo["izquierdo"]
    lejos = nodo["derecho"]
else:
    cerca = nodo["derecho"]
    lejos = nodo["izquierdo"]

buscar(cerca, q, mejor)
if delta * delta < mejor["dist"]:
    buscar(lejos, q, mejor)
```

Mejor a distancia 1: el círculo no cruza el corte. $\delta^2 = 4$ no es menor que 1, la derecha no se revisa.

Mejor a distancia 3: el círculo cruza el corte. $4 < 9$, hay que revisar la derecha.

100 % de las consultas con el mismo vecino que la búsqueda exhaustiva, en todas las d

src/kdtree.py

ACTIVIDAD 4 · RESULTADOS

Más dimensiones: más nodos visitados y mejor accuracy

d	nodos visitados (de 1437)	speedup	accuracy
2	1,2 %	**38,8×**	0,447
4	3,7 %	**14,6×**	0,806
8	14,6 %	**4,5×**	0,931
16	34,3 %	**2,2×**	0,958
32	51,3 %	**1,8×**	0,972

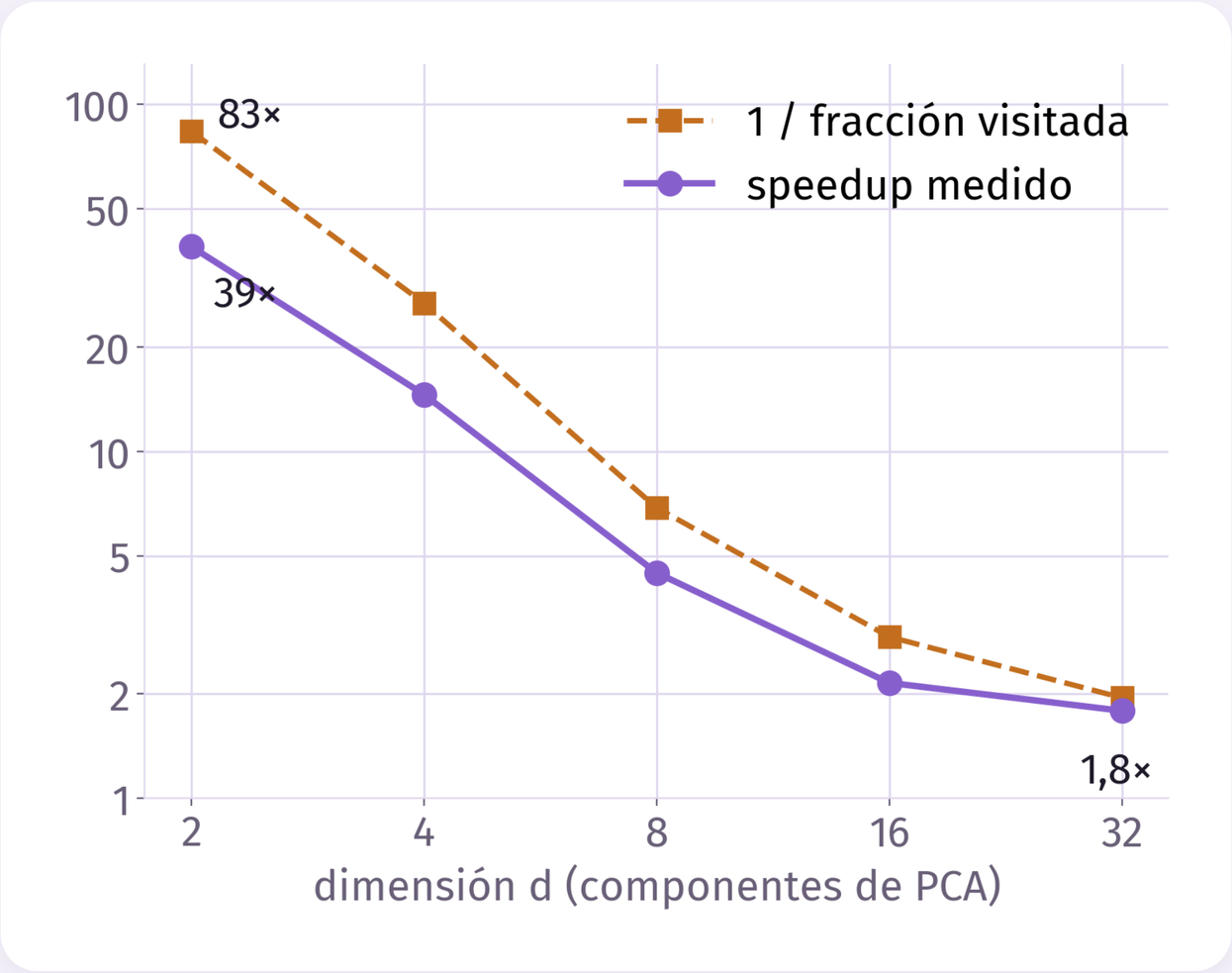
Speedup = tiempo exhaustiva / tiempo k-d tree. Mediana de 7 repeticiones.

DISCUSIÓN (A)

El speedup sigue a 1 / fracción visitada, algo por debajo

Cada nodo visitado agrega la llamada recursiva, el cálculo de δ y la decisión de qué lado explorar.

Con d = 2 el árbol evalúa 83 veces menos distancias y es 39 veces más rápido.



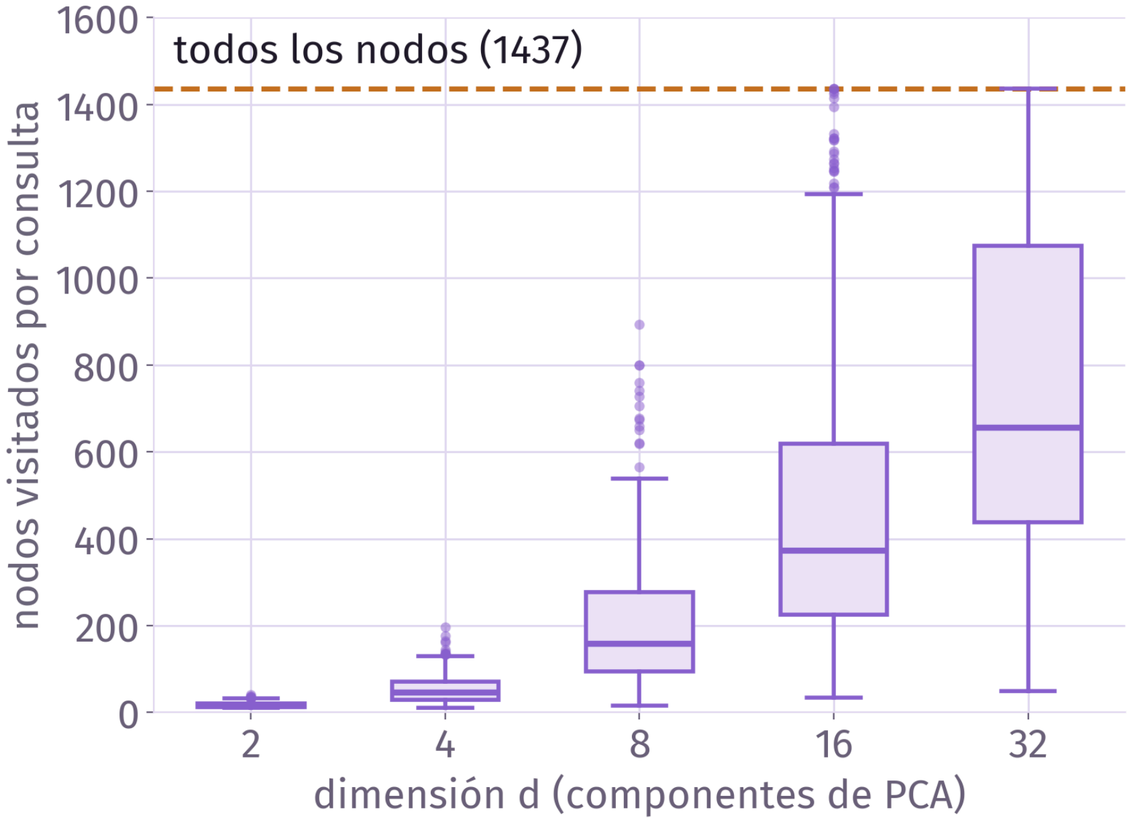
Ejes en escala logarítmica

DISCUSIÓN (B) Y (C)

Con d = 16 y 32 hay consultas que visitan los 1437 nodos

δ mide una coordenada y la distancia al mejor vecino suma d. Con más dimensiones la poda descarta menos.

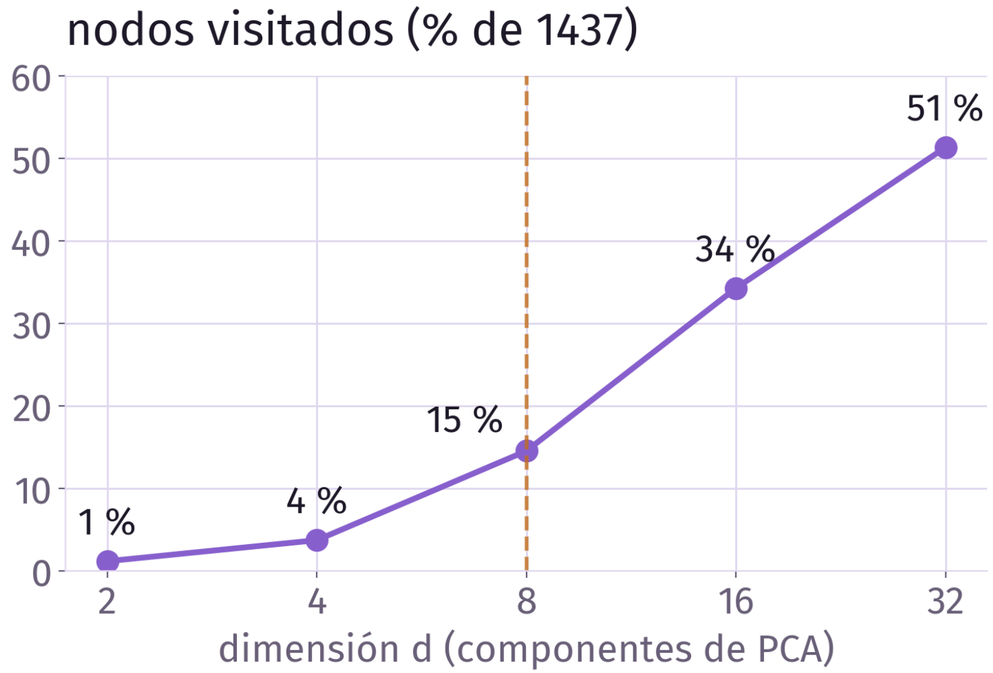
Es consistente con un peor caso $\Theta(n)$ por consulta: la búsqueda es exacta y, si no poda, revisa todo.

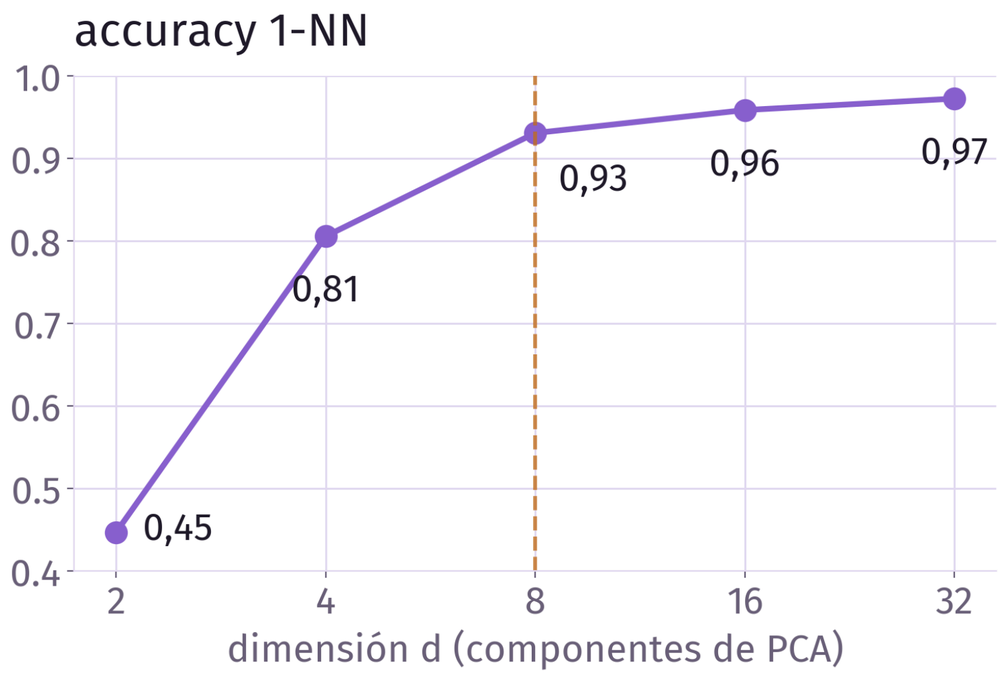


Cada caja va del primer al tercer cuartil

DISCUSIÓN (D)

Reducir d acelera el árbol y empeora la clasificación





Con d = 8 la accuracy es 0,93 y la búsqueda es 4,5 veces más rápida. Las dos búsquedas tienen la misma accuracy en cada d.

ACTIVIDAD 4 · RESULTADOS

Lo que cambia con d y lo que no

≈ 3 ms

Construcción

Casi no cambia con d: el orden compara una sola coordenada.

6,5×

Exhaustiva, de d = 2 a 32

De 0,24 a 1,55 ms por consulta. Cada distancia recorre d coordenadas.

0,967

Accuracy con 64

Menor que con 32 (0,972). Más dimensiones no siempre ayudan.

DISCUSIÓN (E)

El análisis y la medición responden preguntas distintas

Análisis asintótico

Garantías para cualquier entrada: construcción $\Theta(n \log^2 n)$ y consulta $\Theta(n)$ en el peor caso.

Oculta las constantes y no dice cuánto se poda con datos concretos.

Medición

Costo real con estos datos, este n y este computador, incluido el costo extra de la recursión.

Sola no generaliza a otros tamaños ni a otro hardware.

Conclusiones

1 Si las vueltas del ciclo interno dependen del externo, hay que sumarlas. Por eso A es $\Theta(n)$ y B es $\Theta(n \log n)$.

2 En un árbol de recursión, si el trabajo baja en forma geométrica domina la raíz (3.1, $\Theta(n)$). Si baja de a poco, pesan los log n niveles (3.2, $\Theta(n \log^2 n)$).

3 El k-d tree es exacto y muy rápido con d chico, y su peor caso $\Theta(n)$ aparece desde d = 16. El análisis da la garantía y la medición muestra cuánto se poda con datos reales.

Gracias.

Paula Vásquez-Henríquez · Guillermo Zieballe Código, pruebas y experimentos en el repositorio de la tarea

Doctorado en Inteligencia Artificial · CRUCH
Biobío

MATERIAL COMPLEMENTARIO

Anexos

A1 tiempos y trabajo por consulta, con la referencia de 64 componentes.

Tiempos y trabajo por consulta

A1

d	construcción (ms)	exhaustiva / consulta (ms)	k-d tree / consulta (ms)	visitados (mediana)	speedup	accuracy
2	4,6	0,237	0,006	16	38,85	0,4472
4	2,7	0,320	0,022	45	14,57	0,8056
8	2,4	0,487	0,109	158	4,46	0,9306
16	3,3	0,792	0,368	373	2,15	0,9583
32	2,4	1,552	0,868	656	1,79	0,9722
64*	2,4	2,781	2,041	891	1,36	0,9667

* Referencia con las 64 componentes. Apple M2, Python 3.12.2, scikit-learn 1.6.1.